Building a Autoregressive Neural Network

Part 1

Luca WB

2026-01-28

Table of contents

0.1 Brief summary 2
0.2 Setup 2
0.3 Creating a baseline 2

0.1 Brief summary

In this post, we will implement an Autoregressive Neural Network from scratch, relying solely on the PyTorch tensor class. We assume prior familiarity with Neural Networks; however, if your knowledge feels a bit rusty or you need a refresher, I recommend reading this post beforehand Building Neural Networks from Scratch.

The main reason for this is to learn how an Autoregressive NN works to generate words, for this, I'm drawing on Andrej Karpathy's video series about makemore, a network capable of creating more words of the same type, so if you train with names, it generates more proper names it generates more words that remember proper names, and so on with anything that is formed by letters.

In this post, I will cover how to make a simple model for our baseline, and how to implement a model with MLP and compare them.

0.2 Setup

First, you need to download PyTorch and the dataset. For PyTorch, just download in the official site https://pytorch.org/get-started/locally/. Now, for the dataset, you can create your own with random names that you can think, but It's much easier just download the names.txt dataset from the Andrej repository https://github.com/karpathy/makemore/blob/master/names.txt.

0.3 Creating a baseline

In propose of this, it's just to create the most simple and naive model. It's important because we need some baseline to compare with our future models, so we will create a model called bigram, the logic is just to look to the last character. Note that you will use just one character of context for our model, and we will consider that the most small part

of our word is a character, for models like chatGPT, they don't use characters, they use combinations of characters similar to syllables.

So, to start, we need first import our dataset and PyTorch

```
import torch

# Basically makes a list of all the names
names = open("names.txt", "r").read().splitlines()
names[:5]
```

```
['emma', 'olivia', 'ava', 'isabella', 'sophia']
```

Most part of models usually can't handle with characters, so it's useful to convert this letters in numbers in some way. For this, there are many possibles, but I will use just a simple dictionary to convert them. But we

```
chars = sorted(list(set("".join(names)))) # Creates an ordered list with all letters in ou
charToInt = {s:i+1 for i,s in enumerate(chars)} # Creates a dict to convert chars to int,
charToInt["."] = 0 # I will explain later why we need a special character
print(charToInt)
```

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6, 'g': 7, 'h': 8, 'i': 9, 'j': 10, 'k': 11,
```

Just to get it ready, if we convert to int, so we can read it at the end, we will need an intToChar converter, so let's get it ready

```
intToChar = {s:i for i,s in charToInt.items()}
print(intToChar)
```

```
{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e', 6: 'f', 7: 'g', 8: 'h', 9: 'i', 10: 'j', 11: 'k',
```

Know, for our model, we need to calculate the total number that each sequence occurs, like, with we start with letter "a", how many times occurs that "m" is the next character. And it's for this that we need and special characters, because we always need something to start, after all, the autoregressive model logic and take the output of the model and put it in its input, so we need an initial input. In our case, we will use "." as the symbol to start a name/words and to stop word (without a final symbol, it would generate forever). To make more clear, see the code bellow

```
N = torch.zeros((27,27)).int()

for name in names:
    chars = ["."] + list(name) + ["."] # turn the name in a list of characters and add "."
    for ch1,ch2 in zip(chars, chars[1:]): # In each loop, pick up one letter in ch1, and t
        id1, id2 = charToInt[ch1], charToInt[ch2]
        N[id1, id2] += 1
```

Basically, this count how often some sequence of characters occurs, like the most common letter sequence is "n" follow by ".", this mean, that the most commum letter to finish a name in our dataset it's "n". If you run with all the names, you can use the code bellow to find the most common occurrences

```
id1, id2 = (N == N.max()).nonzero(as_tuple=True) # Creates a boolean matrix that only it's
print(intToChar[id1.item()], "-->", intToChar[id2.item()],"occurs ", N.max().item())
```

```
n --> . occurs  6763
```

So let's see how our bigrams are distributed

```
import matplotlib.pyplot as plt

plt.figure(figsize= (16,16))
plt.imshow(N, cmap="Blues")
for i in range(27):
    for j in range(27):
        chstr = intToChar[i] + intToChar[j]
        plt.text(j,i, chstr, ha="center", va="bottom", color="gray")
        plt.text(j,i, N[i,j].item(), ha="center", va="top", color="gray")

plt.axis("off")
```

.. 0	.a 4410	.b 1306	.c 1542	.d 1690	.e 1531	.f 417	.g 669	.h 874	.i 591	.j 2422	.k 2963	.l 1572	.m 2538	.n 1146	.o 394	.p 515	.q 92	.r 1639	.s 2055	.t 1308	.u 78	.v 376	.w 307	.x 134	.y 535	.z 929
a. 6640	aa 556	ab 541	ac 470	ad 1042	ae 692	af 134	ag 168	ah 2332	ai 1650	aj 175	ak 568	al 2528	am 1634	an 5438	ao 63	ap 82	aq 60	ar 3264	as 1118	at 687	au 381	av 834	aw 161	ax 182	ay 2050	az 435
b. 114	ba 321	bb 38	bc 1	bd 65	be 655	bf 0	bg 0	bh 41	bi 217	bj 1	bk 0	bl 103	bm 0	bn 4	bo 105	bp 0	bq 0	br 842	bs 8	bt 2	bu 45	bv 0	bw 0	bx 0	by 83	bz 0
c. 97	ca 815	cb 0	cc 42	cd 1	ce 551	cf 0	cg 2	ch 664	ci 271	cj 3	ck 316	cl 116	cm 0	cn 0	co 380	cp 1	cq 11	cr 76	cs 5	ct 35	cu 35	cv 0	cw 0	cx 3	cy 104	cz 4
d. 516	da 1303	db 1	dc 3	dd 149	de 1283	df 5	dg 25	dh 118	di 674	dj 9	dk 3	dl 60	dm 30	dn 31	do 378	dp 0	dq 1	dr 424	ds 29	dt 4	du 92	dv 17	dw 23	dx 0	dy 317	dz 1
e. 3983	ea 679	eb 121	ec 153	ed 384	ee 1271	ef 82	eg 125	eh 152	ei 818	ej 55	ek 178	el 3248	em 769	en 2675	eo 269	ep 83	eq 14	er 1958	es 861	et 580	eu 69	ev 463	ew 50	ex 132	ey 1070	ez 181
f. 80	fa 242	fb 0	fc 0	fd 0	fe 123	ff 44	fg 1	fh 1	fi 160	fj 0	fk 2	fl 20	fm 0	fn 4	fo 60	fp 0	fq 0	fr 114	fs 6	ft 18	fu 10	fv 0	fw 4	fx 0	fy 14	fz 2
g. 108	ga 330	gb 3	gc 0	gd 19	ge 334	gf 1	gg 25	gh 360	gi 190	gj 3	gk 0	gl 32	gm 6	gn 27	go 83	gp 0	gq 0	gr 201	gs 30	gt 31	gu 85	gv 1	gw 26	gx 0	gy 31	gz 1
h. 2409	ha 2244	hb 8	hc 2	hd 24	he 674	hf 2	hg 2	hh 1	hi 729	hj 9	hk 29	hl 185	hm 117	hn 138	ho 287	hp 1	hq 1	hr 204	hs 31	ht 71	hu 166	hv 39	hw 10	hx 0	hy 213	hz 20
i. 2489	ia 2445	ib 110	ic 509	id 440	ie 1653	if 101	ig 428	ih 95	ii 82	ij 76	ik 445	il 1345	im 427	in 2126	io 588	ip 53	iq 52	ir 849	is 1316	it 541	iu 109	iv 269	iw 8	ix 89	iy 779	iz 277
j. 71	ja 1473	jb 1	jc 4	jd 4	je 440	jf 0	jg 0	jh 45	ji 119	jj 2	jk 2	jl 9	jm 5	jn 2	jo 479	jp 1	jq 0	jr 11	js 7	jt 2	ju 202	jv 5	jw 6	jx 0	jy 10	jz 0
k. 363	ka 1731	kb 2	kc 2	kd 2	ke 895	kf 1	kg 0	kh 307	ki 509	kj 2	kk 20	kl 139	km 9	kn 26	ko 344	kp 0	kq 0	kr 109	ks 95	kt 17	ku 50	kv 2	kw 34	kx 0	ky 379	kz 2
l. 1314	la 2623	lb 52	lc 25	ld 138	le 2921	lf 22	lg 6	lh 19	li 2480	lj 6	lk 24	ll 1345	lm 60	ln 14	lo 692	lp 15	lq 3	lr 18	ls 94	lt 77	lu 324	lv 72	lw 16	lx 0	ly 1588	lz 10
m. 516	ma 2590	mb 112	mc 51	md 24	me 818	mf 1	mg 0	mh 5	mi 1256	mj 7	mk 1	ml 5	mm 168	mn 20	mo 452	mp 38	mq 0	mr 97	ms 35	mt 4	mu 139	mv 3	mw 2	mx 0	my 287	mz 11
n. 6763	na 2977	nb 8	nc 213	nd 704	ne 1359	nf 11	ng 273	nh 26	ni 1725	nj 44	nk 58	nl 195	nm 19	nn 1906	no 496	np 5	nq 2	nr 44	ns 278	nt 443	nu 96	nv 55	nw 11	nx 6	ny 465	nz 145
o. 855	oa 149	ob 140	oc 114	od 190	oe 132	of 34	og 44	oh 171	oi 69	oj 16	ok 68	ol 619	om 261	on 2411	oo 115	op 95	oq 3	or 1059	os 504	ot 118	ou 275	ov 176	ow 114	ox 45	oy 103	oz 54
p. 33	pa 209	pb 2	pc 1	pd 0	pe 197	pf 1	pg 0	ph 204	pi 61	pj 1	pk 1	pl 16	pm 1	pn 1	po 59	pp 39	pq 0	pr 151	ps 16	pt 17	pu 4	pv 0	pw 0	px 0	py 12	pz 0
q. 28	qa 13	qb 0	qc 0	qd 0	qe 1	qf 0	qg 0	qh 0	qi 13	qj 0	qk 0	ql 1	qm 2	qn 0	qo 2	qp 0	qq 0	qr 1	qs 2	qt 0	qu 206	qv 0	qw 3	qx 0	qy 0	qz 0
r. 1377	ra 2356	rb 41	rc 99	rd 187	re 1697	rf 9	rg 76	rh 121	ri 3033	rj 25	rk 90	rl 413	rm 162	rn 140	ro 869	rp 14	rq 16	rr 425	rs 190	rt 208	ru 252	rv 80	rw 21	rx 3	ry 773	rz 23
s. 1169	sa 1201	sb 21	sc 60	sd 9	se 884	sf 2	sg 2	sh 1285	si 684	sj 2	sk 82	sl 279	sm 90	sn 24	so 531	sp 51	sq 1	sr 55	ss 461	st 765	su 185	sv 14	sw 24	sx 0	sy 215	sz 10
t. 483	ta 1027	tb 1	tc 17	td 0	te 716	tf 2	tg 2	th 647	ti 532	tj 3	tk 0	tl 134	tm 4	tn 22	to 667	tp 0	tq 0	tr 352	ts 35	tt 374	tu 78	tv 15	tw 11	tx 2	ty 341	tz 105
u. 155	ua 163	ub 103	uc 103	ud 136	ue 169	uf 19	ug 47	uh 58	ui 121	uj 14	uk 93	ul 301	um 154	un 275	uo 10	up 16	uq 10	ur 414	us 474	ut 82	uu 3	uv 37	uw 86	ux 34	uy 13	uz 45
v. 88	va 642	vb 1	vc 0	vd 1	ve 568	vf 0	vg 0	vh 1	vi 911	vj 0	vk 3	vl 14	vm 0	vn 8	vo 153	vp 0	vq 0	vr 48	vs 0	vt 0	vu 7	vv 7	vw 0	vx 0	vy 121	vz 0
w. 51	wa 280	wb 1	wc 0	wd 8	we 149	wf 2	wg 1	wh 23	wi 148	wj 0	wk 6	wl 13	wm 2	wn 58	wo 36	wp 0	wq 0	wr 22	ws 20	wt 8	wu 25	wv 0	ww 2	wx 0	wy 73	wz 1
x. 164	xa 103	xb 1	xc 4	xd 5	xe 36	xf 3	xg 0	xh 1	xi 102	xj 0	xk 0	xl 39	xm 1	xn 1	xo 41	xp 0	xq 0	xr 0	xs 31	xt 70	xu 5	xv 0	xw 3	xx 38	xy 30	xz 19
y. 2007	ya 2143	yb 27	yc 115	yd 272	ye 301	yf 12	yg 30	yh 22	yi 192	yj 23	yk 86	yl 1104	ym 148	yn 1826	yo 271	yp 15	yq 6	yr 291	ys 401	yt 104	yu 141	yv 106	yw 4	yx 28	yy 23	yz 78
z. 160	za 860	zb 4	zc 2	zd 2	ze 373	zf 0	zg 1	zh 43	zi 364	zj 2	zk 2	zl 123	zm 35	zn 4	zo 110	zp 2	zq 0	zr 32	zs 4	zt 4	zu 73	zv 2	zw 3	zx 1	zy 147	zz 45

One thing very interesting you can note, it's that have many combinations that don't exist, like "bk" or "gc". This makes it impossible for our model to generate a name with this combination, it is ok to leave it like this, but it would be a good practice to add 1 in all values, thus ensuring that at least there is the minimal possibility of generating a rare sequence

```
N = N + 1
```

.. 1	.a 4411	.b 1307	.c 1543	.d 1691	.e 1532	.f 418	.g 670	.h 875	.i 592	.j 2423	.k 2964	.l 1573	.m 2539	.n 1147	.o 395	.p 516	.q 93	.r 1640	.s 2056	.t 1309	.u 79	.v 377	.w 308	.x 135	.y 536	.z 930
a. 6641	aa 557	ab 542	ac 471	ad 1043	ae 693	af 135	ag 169	ah 2333	ai 1651	aj 176	ak 569	al 2529	am 1635	an 5439	ao 64	ap 83	aq 61	ar 3265	as 1119	at 688	au 382	av 835	aw 162	ax 183	ay 2051	az 436
b. 115	ba 322	bb 39	bc 2	bd 66	be 656	bf 1	bg 1	bh 42	bi 218	bj 2	bk 1	bl 104	bm 1	bn 5	bo 106	bp 1	bq 1	br 843	bs 9	bt 3	bu 46	bv 1	bw 1	bx 1	by 84	bz 1
c. 98	ca 816	cb 1	cc 43	cd 2	ce 552	cf 1	cg 3	ch 665	ci 272	cj 4	ck 317	cl 117	cm 1	cn 1	co 381	cp 2	cq 12	cr 77	cs 6	ct 36	cu 36	cv 1	cw 1	cx 4	cy 105	cz 5
d. 517	da 1304	db 2	dc 4	dd 150	de 1284	df 6	dg 26	dh 119	di 675	dj 10	dk 4	dl 61	dm 31	dn 32	do 379	dp 1	dq 2	dr 425	ds 30	dt 5	du 93	dv 18	dw 24	dx 1	dy 318	dz 2
e. 3984	ea 680	eb 122	ec 154	ed 385	ee 1272	ef 83	eg 126	eh 153	ei 819	ej 56	ek 179	el 3249	em 770	en 2676	eo 270	ep 84	eq 15	er 1959	es 862	et 581	eu 70	ev 464	ew 51	ex 133	ey 1071	ez 182
f. 81	fa 243	fb 1	fc 1	fd 1	fe 124	ff 45	fg 2	fh 2	fi 161	fj 1	fk 3	fl 21	fm 1	fn 5	fo 61	fp 1	fq 1	fr 115	fs 7	ft 19	fu 11	fv 1	fw 5	fx 1	fy 15	fz 3
g. 109	ga 331	gb 4	gc 1	gd 20	ge 335	gf 2	gg 26	gh 361	gi 191	gj 4	gk 1	gl 33	gm 7	gn 28	go 84	gp 1	gq 1	gr 202	gs 31	gt 32	gu 86	gv 2	gw 27	gx 1	gy 32	gz 2
h. 2410	ha 2245	hb 9	hc 3	hd 25	he 675	hf 3	hg 3	hh 2	hi 730	hj 10	hk 30	hl 186	hm 118	hn 139	ho 288	hp 2	hq 2	hr 205	hs 32	ht 72	hu 167	hv 40	hw 11	hx 1	hy 214	hz 21
i. 2490	ia 2446	ib 111	ic 510	id 441	ie 1654	if 102	ig 429	ih 96	ii 83	ij 77	ik 446	il 1346	im 428	in 2127	io 589	ip 54	iq 53	ir 850	is 1317	it 542	iu 110	iv 270	iw 9	ix 90	iy 780	iz 278
j. 72	ja 1474	jb 2	jc 5	jd 5	je 441	jf 1	jg 1	jh 46	ji 120	jj 3	jk 3	jl 10	jm 6	jn 3	jo 480	jp 2	jq 1	jr 12	js 8	jt 3	ju 203	jv 6	jw 7	jx 1	jy 11	jz 1
k. 364	ka 1732	kb 3	kc 3	kd 3	ke 896	kf 2	kg 1	kh 308	ki 510	kj 3	kk 21	kl 140	km 10	kn 27	ko 345	kp 1	kq 1	kr 110	ks 96	kt 18	ku 51	kv 3	kw 35	kx 1	ky 380	kz 3
l. 1315	la 2624	lb 53	lc 26	ld 139	le 2922	lf 23	lg 7	lh 20	li 2481	lj 7	lk 25	ll 1346	lm 61	ln 15	lo 693	lp 16	lq 4	lr 19	ls 95	lt 78	lu 325	lv 73	lw 17	lx 1	ly 1589	lz 11
m. 517	ma 2591	mb 113	mc 52	md 25	me 819	mf 2	mg 1	mh 6	mi 1257	mj 8	mk 2	ml 6	mm 169	mn 21	mo 453	mp 39	mq 1	mr 98	ms 36	mt 5	mu 140	mv 4	mw 3	mx 1	my 288	mz 12
n. 6764	na 2978	nb 9	nc 214	nd 705	ne 1360	nf 12	ng 274	nh 27	ni 1726	nj 45	nk 59	nl 196	nm 20	nn 1907	no 497	np 6	nq 3	nr 45	ns 279	nt 444	nu 97	nv 56	nw 12	nx 7	ny 466	nz 146
o. 856	oa 150	ob 141	oc 115	od 191	oe 133	of 35	og 45	oh 172	oi 70	oj 17	ok 69	ol 620	om 262	on 2412	oo 116	op 96	oq 4	or 1060	os 505	ot 119	ou 276	ov 177	ow 115	ox 46	oy 104	oz 55
p. 34	pa 210	pb 3	pc 2	pd 1	pe 198	pf 2	pg 1	ph 205	pi 62	pj 2	pk 2	pl 17	pm 2	pn 2	po 60	pp 40	pq 1	pr 152	ps 17	pt 18	pu 5	pv 1	pw 1	px 1	py 13	pz 1
q. 29	qa 14	qb 1	qc 1	qd 1	qe 2	qf 1	qg 1	qh 1	qi 14	qj 1	qk 1	ql 2	qm 3	qn 1	qo 3	qp 1	qq 1	qr 2	qs 3	qt 1	qu 207	qv 1	qw 4	qx 1	qy 1	qz 1
r. 1378	ra 2357	rb 42	rc 100	rd 188	re 1698	rf 10	rg 77	rh 122	ri 3034	rj 26	rk 91	rl 414	rm 163	rn 141	ro 870	rp 15	rq 17	rr 426	rs 191	rt 209	ru 253	rv 81	rw 22	rx 4	ry 774	rz 24
s. 1170	sa 1202	sb 22	sc 61	sd 10	se 885	sf 3	sg 3	sh 1286	si 685	sj 3	sk 83	sl 280	sm 91	sn 25	so 532	sp 52	sq 2	sr 56	ss 462	st 766	su 186	sv 15	sw 25	sx 1	sy 216	sz 11
t. 484	ta 1028	tb 2	tc 18	td 1	te 717	tf 3	tg 3	th 648	ti 533	tj 4	tk 1	tl 135	tm 5	tn 23	to 668	tp 1	tq 1	tr 353	ts 36	tt 375	tu 79	tv 16	tw 12	tx 3	ty 342	tz 106
u. 156	ua 164	ub 104	uc 104	ud 137	ue 170	uf 20	ug 48	uh 59	ui 122	uj 15	uk 94	ul 302	um 155	un 276	uo 11	up 17	uq 11	ur 415	us 475	ut 83	uu 4	uv 38	uw 87	ux 35	uy 14	uz 46
v. 89	va 643	vb 2	vc 1	vd 2	ve 569	vf 1	vg 1	vh 2	vi 912	vj 1	vk 4	vl 15	vm 1	vn 9	vo 154	vp 1	vq 1	vr 49	vs 1	vt 1	vu 8	vv 8	vw 1	vx 1	vy 122	vz 1
w. 52	wa 281	wb 2	wc 1	wd 9	we 150	wf 3	wg 2	wh 24	wi 149	wj 1	wk 7	wl 14	wm 3	wn 59	wo 37	wp 1	wq 1	wr 23	ws 21	wt 9	wu 26	wv 1	ww 3	wx 1	wy 74	wz 2
x. 165	xa 104	xb 2	xc 5	xd 6	xe 37	xf 4	xg 1	xh 2	xi 103	xj 1	xk 1	xl 40	xm 2	xn 2	xo 42	xp 1	xq 1	xr 1	xs 32	xt 71	xu 6	xv 1	xw 4	xx 39	xy 31	xz 20
y. 2008	ya 2144	yb 28	yc 116	yd 273	ye 302	yf 13	yg 31	yh 23	yi 193	yj 24	yk 87	yl 1105	ym 149	yn 1827	yo 272	yp 16	yq 7	yr 292	ys 402	yt 105	yu 142	yv 107	yw 5	yx 29	yy 24	yz 79
z. 161	za 861	zb 5	zc 3	zd 3	ze 374	zf 1	zg 2	zh 44	zi 365	zj 3	zk 3	zl 124	zm 36	zn 5	zo 111	zp 3	zq 1	zr 33	zs 5	zt 5	zu 74	zv 3	zw 4	zx 2	zy 148	zz 46

So, lets transform our probability matrix

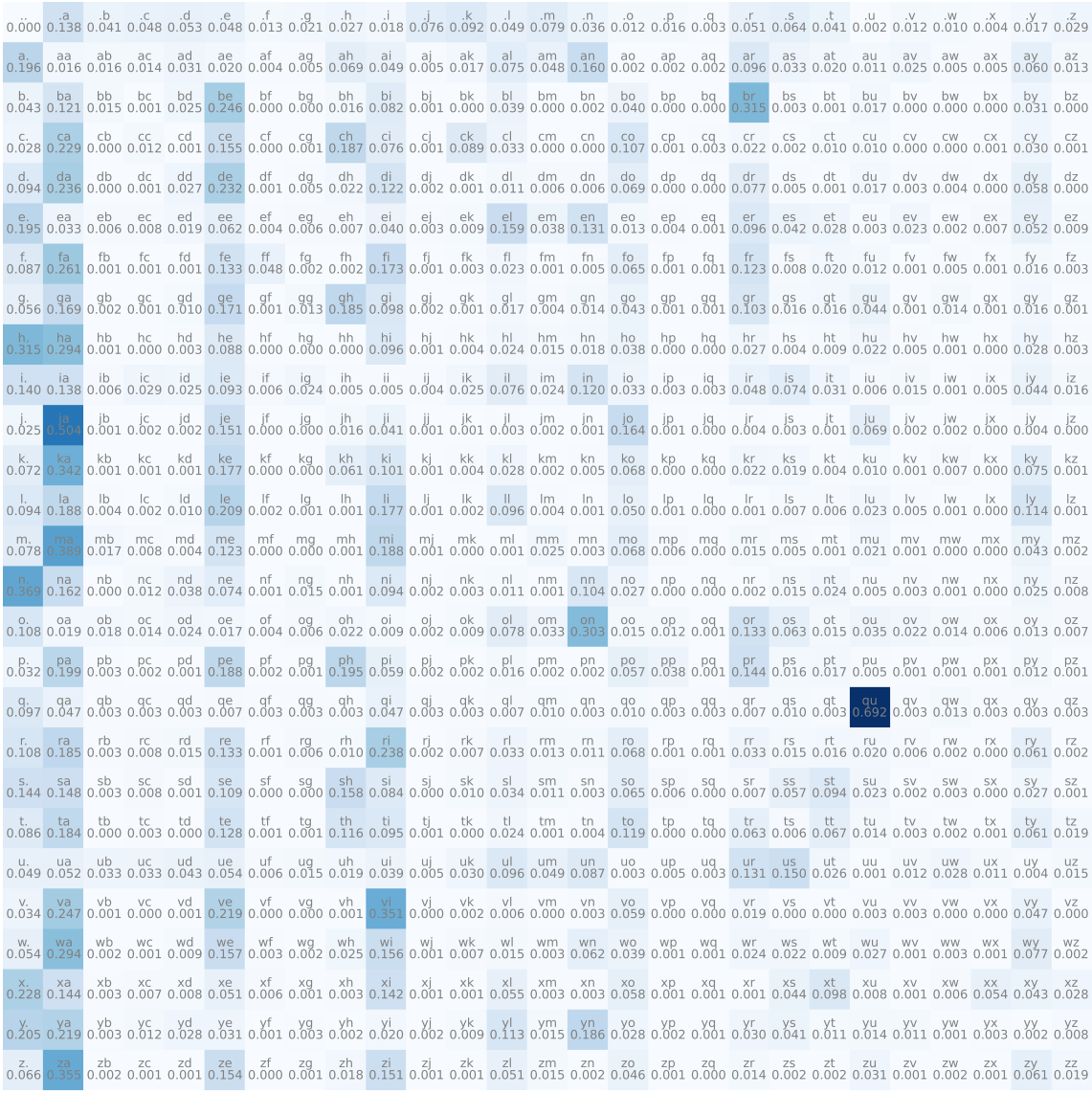
```
P = N
P = P / P.sum(dim=1, keepdims=True)
```

```
import matplotlib.pyplot as plt

plt.figure(figsize= (16,16))
plt.imshow(P, cmap="Blues")
for i in range(27):
    for j in range(27):
        chstr = intToChar[i] + intToChar[j]
        plt.text(j,i, chstr, ha="center", va="bottom", color="gray")
```

```
        plt.text(j,i, f"{P[i,j].item():.3f}", ha="center", va="top", color="gray")

plt.axis("off")
```



Some probabilities stay in 0 because the visualization it's limited to 3 decimal numbers. Now we already have our model, it's just our probability matrix P, bellow I will show how to use it.

```
import random

for i in range(10):
    out = []
```

```python
    init = 0
    while True:
        id = torch.multinomial(P[init], num_samples=1, replacement=True).item()

        if id == 0:
            break

        out.append(intToChar[id])
        init = id
    print("".join(out))
```

```
ladhai
ken
ile
chiliariali
jamitt
janany
h
sekal
trlelerilynemi
llsdrgaabr
```